F1 Racing Stats Web Application

Name: Jugal Patel
Enr.No: 24162581032
Sub: FET

A full-stack Formula 1 racing statistics web application built with Angular, Node.js, Express, and MongoDB. The application provides a comprehensive REST API for accessing F1 racing data including drivers, constructors, circuits, races, results, and championship standings.

Table of Contents

- [Tech Stack](#tech-stack)
- [Project Structure](#project-structure)
- [Database Models](#database-models)
- [API Endpoints](#api-endpoints)
- [Source Code](#source-code)
- [How It Was Made](#how-it-was-made)
- [Getting Started](#getting-started)
- [Future Enhancements](#future-enhancements)

Tech Stack

Layer	Technology
Frontend	Angular (TypeScript)
Backend	Node.js + Express.js
Database	MongoDB with Mongoose ODM
Data Import	CSV files via csvtojson
Logging	Winston
HTTP Logging	Morgan

Project Structure

```
f1/
├── archive/                    # CSV data files
│   ├── drivers.csv
│   ├── constructors.csv
│   ├── circuits.csv
│   ├── races.csv
│   ├── results.csv
│   ├── driver_standings.csv
│   └── ...
├── backend/
│   ├── server.js             # Main Express server
│   ├── models/               # Mongoose schemas
│   ├── import/               # CSV import script
│   └── utils/                # Logger configuration
├── frontend/
│   └── src/
```

```
│       ├── main.ts           # Angular entry point
│       └── styles.scss      # Styles
├── images/                   # Car images & logos
└── package.json
```

Database Models

Driver Model (`backend/models/Driver.js`)

```javascript
const mongoose = require('mongoose');

const driverSchema = new mongoose.Schema({
  driverId: { type: Number, unique: true },
  driverRef: String,
  number: Number,
  code: String,
  forename: String,
  surname: String,
  dob: Date,
  nationality: String,
  url: String
});

module.exports = mongoose.model('Driver', driverSchema);
```

Race Model (`backend/models/Race.js`)

```javascript
const mongoose = require('mongoose');

const raceSchema = new mongoose.Schema({
  raceId: { type: Number, unique: true },
  year: Number,
  round: Number,
  circuitId: Number,
  name: String,
  date: Date,
  time: String,
  url: String,
  fp1_date: Date,
  fp1_time: String,
  fp2_date: Date,
  fp2_time: String,
  fp3_date: Date,
  fp3_time: String,
  quali_date: Date,
  quali_time: String,
  sprint_date: Date,
  sprint_time: String
});

module.exports = mongoose.model('Race', raceSchema);
```

Result Model (`backend/models/Result.js`)

```javascript
const mongoose = require('mongoose');

const resultSchema = new mongoose.Schema({
  resultId: { type: Number, unique: true },
  raceId: Number,
  driverId: Number,
  constructorId: Number,
  number: Number,
  grid: Number,
  position: Number,
  positionText: String,
  positionOrder: Number,
  points: Number,
  laps: Number,
  time: String,
  milliseconds: Number,
  fastestLap: Number,
  rank: Number,
  fastestLapTime: String,
  fastestLapSpeed: String,
  statusId: Number
});

module.exports = mongoose.model('Result', resultSchema);
```

API Endpoints

Health & Info

Method	Endpoint	Description
GET	`/api/health`	Health check with MongoDB status
GET	`/api/info`	API information and endpoints

Drivers

Method	Endpoint	Description
GET	`/api/drivers`	List all drivers
GET	`/api/drivers/:id`	Get driver by ID
GET	`/api/drivers/stats`	Get driver statistics (wins, podiums)

Constructors

Method	Endpoint	Description
GET	`/api/constructors`	List all constructors
GET	`/api/constructors/:id`	Get constructor by ID

Circuits

Method	Endpoint	Description
GET	`/api/circuits`	List all circuits
GET	`/api/circuits/:id`	Get circuit by ID

Races & Results

Method	Endpoint	Description
GET	`/api/races`	List all races (supports `?year=2024`)
GET	`/api/races/:id`	Get race by ID
GET	`/api/results/:raceId`	Get results for a specific race
GET	`/api/latest-results`	Get results for the latest race
GET	`/api/standings/:year`	Get driver standings for a year
GET	`/api/champions`	Get current championship standings

Source Code

Main Server (`backend/server.js`)

```javascript
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const morgan = require('morgan');
const path = require('path');

const { logger, apiLogger, dbLogger } = require('./utils/logger');

const Driver = require('./models/Driver');
const Constructor = require('./models/Constructor');
const Circuit = require('./models/Circuit');
const Race = require('./models/Race');
const Result = require('./models/Result');
const DriverStanding = require('./models/DriverStanding');

const app = express();

app.use(cors());
app.use(express.json());

app.use(morgan('combined', {
  stream: {
    write: (message) => apiLogger.http(message.trim())
  }
}));

const PORT = process.env.PORT || 3000;
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://localhost:27017/f1db';

mongoose.connect(MONGO_URI, {
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 45000,
})
  .then(() => logger.info('MongoDB connection established'))
```

```
  .catch(err => logger.error('MongoDB connection failed:', { error:
err.message, stack: err.stack }));

app.get('/api/health', (req, res) => {
  const healthcheck = {
    uptime: process.uptime(),
    message: 'OK',
    timestamp: Date.now(),
    mongodb: mongoose.connection.readyState === 1 ? 'connected' :
'disconnected',
    environment: process.env.NODE_ENV || 'development'
  };

  try {
    res.status(200).json(healthcheck);
  } catch (error) {
    healthcheck.message = error.message;
    res.status(503).json(healthcheck);
  }
});

app.get('/api/drivers', async (req, res) => {
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/drivers`, { query: req.query
});

  try {
    const startTime = Date.now();
    const drivers = await Driver.find();
    const duration = Date.now() - startTime;

    apiLogger.info(`[${requestId}] Retrieved ${drivers.length} drivers`,
{ duration: `${duration}ms` });

    res.status(200).json({
      success: true,
      count: drivers.length,
      data: drivers,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching drivers`, {
      error: err.message,
      stack: err.stack
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch drivers',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});

app.get('/api/drivers/:id', async (req, res) => {
```

```
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/drivers/${req.params.id}`);

  try {
    const startTime = Date.now();
    const driver = await Driver.findOne({ driverId: req.params.id });
    const duration = Date.now() - startTime;

    if (!driver) {
      apiLogger.warn(`[${requestId}] Driver not found`, { id:
req.params.id });

      return res.status(404).json({
        success: false,
        error: 'Driver not found',
        message: `No driver found with ID ${req.params.id}`,
        timestamp: new Date().toISOString(),
        requestId
      });
    }

    res.status(200).json({
      success: true,
      data: driver,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching driver`, {
      error: err.message,
      stack: err.stack,
      id: req.params.id
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch driver',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});

app.get('/api/races', async (req, res) => {
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/races`, { query: req.query });

  try {
    const startTime = Date.now();
    const { year, limit } = req.query;
    const filter = year ? { year: parseInt(year) } : {};
    const races = await Race.find(filter)
      .sort({ date: -1 })
      .limit(limit ? parseInt(limit) : 0);
    const duration = Date.now() - startTime;

    res.status(200).json({
```

```
      success: true,
      count: races.length,
      data: races,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching races`, {
      error: err.message,
      stack: err.stack
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch races',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});

app.get('/api/results/:raceId', async (req, res) => {
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/results/${req.params.raceId}`);

  try {
    const startTime = Date.now();
    const results = await Result.find({ raceId: req.params.raceId })
      .sort({ positionOrder: 1 })
      .populate('driverId')
      .populate('constructorId');
    const duration = Date.now() - startTime;

    res.status(200).json({
      success: true,
      count: results.length,
      data: results,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching results`, {
      error: err.message,
      stack: err.stack,
      raceId: req.params.raceId
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch results',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});
```

```
app.get('/api/latest-results', async (req, res) => {
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/latest-results`);

  try {
    const startTime = Date.now();
    const latestRace = await Race.findOne().sort({ date: -1 });

    if (!latestRace) {
      return res.status(404).json({
        success: false,
        error: 'No races found',
        message: 'No race data available in the database',
        timestamp: new Date().toISOString(),
        requestId
      });
    }

    const results = await Result.find({ raceId: latestRace.raceId })
      .sort({ positionOrder: 1 })
      .populate('driverId')
      .populate('constructorId');

    const duration = Date.now() - startTime;

    res.status(200).json({
      success: true,
      race: latestRace,
      results,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching latest results`, {
      error: err.message,
      stack: err.stack
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch latest results',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});

app.get('/api/standings/:year', async (req, res) => {
  const requestId = Date.now().toString(36);
  apiLogger.info(`[${requestId}] GET /api/standings/${req.params.year}`);

  try {
    const startTime = Date.now();
    const year = parseInt(req.params.year);
    const latestRace = await Race.findOne({ year }).sort({ date: -1 });

    if (!latestRace) {
```

```
      return res.status(404).json({
        success: false,
        error: 'No races found',
        message: `No races found for year ${year}`,
        timestamp: new Date().toISOString(),
        requestId
      });
    }

    const standings = await DriverStanding.find({ raceId:
latestRace.raceId })
      .sort({ position: 1 })
      .populate('driverId');

    const duration = Date.now() - startTime;

    res.status(200).json({
      success: true,
      year,
      race: latestRace.name,
      standings,
      timestamp: new Date().toISOString(),
      requestId
    });
  } catch (err) {
    apiLogger.error(`[${requestId}] Error fetching standings`, {
      error: err.message,
      stack: err.stack,
      year: req.params.year
    });

    res.status(500).json({
      success: false,
      error: 'Failed to fetch standings',
      message: err.message,
      timestamp: new Date().toISOString(),
      requestId
    });
  }
});

if (process.env.NODE_ENV === 'production') {
  app.use(express.static(path.join(__dirname,
'../frontend/dist/frontend')));
  app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname,
'../frontend/dist/frontend/index.html'));
  });
}

app.listen(PORT, () => {
  logger.info(`
```

```
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║   🚀 F1 API Server Started                                ║
║                                                           ║
║   Environment: ${(process.env.NODE_ENV || 'development').padEnd(36)}║
║   Port: ${PORT.toString().padEnd(46)}║
║   MongoDB: ${MONGO_URI.substring(0, 30).padEnd(46)}║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
  `);
});

module.exports = app;
```

CSV Import Script (`backend/import/csvImport.js`)

```javascript
const csvtojson = require('csvtojson');
const mongoose = require('mongoose');
const path = require('path');
const fs = require('fs');

const { logger, importLogger, dbLogger } = require('../utils/logger');

const Driver = require('../models/Driver');
const Constructor = require('../models/Constructor');
const Circuit = require('../models/Circuit');
const Race = require('../models/Race');
const Result = require('../models/Result');
const DriverStanding = require('../models/DriverStanding');

const MONGO_URI = process.env.MONGO_URI ||
'mongodb://localhost:27017/f1db';
const CSV_DIR = process.env.CSV_DIR || path.join(__dirname,
'../../archive');

function convertNull(value) {
  return value === '\\N' ? null : value;
}

function convertNumber(value) {
  if (value === '\\N' || value === null || value === '') return null;
  const num = parseInt(value);
  return isNaN(num) ? null : num;
}

function convertFloat(value) {
  if (value === '\\N' || value === null || value === '') return null;
  const num = parseFloat(value);
  return isNaN(num) ? null : num;
}

function convertDate(value) {
  if (value === '\\N' || !value) return null;
  try {
    return new Date(value);
  } catch (err) {
```

```
    importLogger.warn(`Date conversion error: ${value}`, { error:
err.message });
    return null;
  }
}

async function importDrivers() {
  const startTime = Date.now();
  const filePath = path.join(CSV_DIR, 'drivers.csv');

  importLogger.info('Starting drivers import...', { file: filePath });

  const json = await csvtojson().fromFile(filePath);
  const drivers = [];

  for (let i = 0; i < json.length; i++) {
    const row = json[i];
    drivers.push({
      driverId: convertNumber(row.driverId),
      driverRef: convertNull(row.driverRef),
      number: convertNumber(row.number),
      code: convertNull(row.code),
      forename: convertNull(row.forename),
      surname: convertNull(row.surname),
      dob: convertDate(row.dob),
      nationality: convertNull(row.nationality),
      url: convertNull(row.url)
    });
  }

  await Driver.deleteMany({});
  if (drivers.length > 0) {
    await Driver.insertMany(drivers);
  }

  const duration = Date.now() - startTime;
  importLogger.info(`✅ Drivers import complete!`, { imported:
drivers.length, totalTime: duration });
}

async function importRaces() {
  const startTime = Date.now();
  const filePath = path.join(CSV_DIR, 'races.csv');

  const json = await csvtojson().fromFile(filePath);
  const races = [];

  for (let i = 0; i < json.length; i++) {
    const row = json[i];
    races.push({
      raceId: convertNumber(row.raceId),
      year: convertNumber(row.year),
      round: convertNumber(row.round),
      circuitId: convertNumber(row.circuitId),
      name: convertNull(row.name),
      date: convertDate(row.date),
      time: convertNull(row.time),
      url: convertNull(row.url),
```

```
      fp1_date: convertDate(row.fp1_date),
      fp1_time: convertNull(row.fp1_time),
      fp2_date: convertDate(row.fp2_date),
      fp2_time: convertNull(row.fp2_time),
      fp3_date: convertDate(row.fp3_date),
      fp3_time: convertNull(row.fp3_time),
      quali_date: convertDate(row.quali_date),
      quali_time: convertNull(row.quali_time),
      sprint_date: convertDate(row.sprint_date),
      sprint_time: convertNull(row.sprint_time)
    });
  }

  await Race.deleteMany({});
  if (races.length > 0) {
    await Race.insertMany(races);
  }

  const duration = Date.now() - startTime;
  importLogger.info(`✅ Races import complete!`, { imported:
races.length, totalTime: duration });
}

async function importResults() {
  const startTime = Date.now();
  const filePath = path.join(CSV_DIR, 'results.csv');

  const json = await csvtojson().fromFile(filePath);
  const results = [];
  const batchSize = 5000;

  for (let i = 0; i < json.length; i++) {
    const row = json[i];
    results.push({
      resultId: convertNumber(row.resultId),
      raceId: convertNumber(row.raceId),
      driverId: convertNumber(row.driverId),
      constructorId: convertNumber(row.constructorId),
      number: convertNumber(row.number),
      grid: convertNumber(row.grid),
      position: convertNumber(row.position),
      positionText: convertNull(row.positionText),
      positionOrder: convertNumber(row.positionOrder),
      points: convertFloat(row.points),
      laps: convertNumber(row.laps),
      time: convertNull(row.time),
      milliseconds: convertNumber(row.milliseconds),
      fastestLap: convertNumber(row.fastestLap),
      rank: convertFloat(row.rank),
      fastestLapTime: convertNull(row.fastestLapTime),
      fastestLapSpeed: convertFloat(row.fastestLapSpeed),
      statusId: convertNumber(row.statusId)
    });
  }

  await Result.deleteMany({});

  for (let i = 0; i < results.length; i += batchSize) {
```

```
    const batch = results.slice(i, i + batchSize);
    await Result.insertMany(batch);
    importLogger.info(`Inserted batch ${Math.floor(i / batchSize) +
1}/${Math.ceil(results.length / batchSize)}`);
  }

  const duration = Date.now() - startTime;
  importLogger.info(`✅ Results import complete!`, { imported:
results.length, totalTime: duration });
}

async function run() {
  importLogger.info('🏁 F1 CSV Data Import Starting');

  await mongoose.connect(MONGO_URI, {
    serverSelectionTimeoutMS: 10000,
    socketTimeoutMS: 45000
  });

  importLogger.info('Connecting to MongoDB...');
  console.log('  ✅ Connected to MongoDB\n');

  await importDrivers();
  await importConstructors();
  await importCircuits();
  await importRaces();
  await importResults();
  await importDriverStandings();

  await mongoose.connection.close();
  importLogger.info('✅ All data imported successfully!');

  process.exit(0);
}

run();
```

How It Was Made

This section provides a step-by-step guide on how the F1 application was developed.

Step 1: Project Setup and Initialization

First, create the project directory structure:

```bash
mkdir f1-project
cd f1-project
mkdir -p backend/models backend/import backend/utils backend/utils/logs
frontend/src
mkdir -p archive images/cars images/logos
```

Initialize the Node.js backend:

```bash
cd backend
npm init -y
npm install express mongoose cors morgan csvtojson
```

Step 2: Design Database Models

Create Mongoose schemas for each data entity:

1. **Driver** - stores driver information
2. **Constructor** - stores team information
3. **Circuit** - stores track information
4. **Race** - stores race event details
5. **Result** - stores race results
6. **DriverStanding** - stores championship positions

Step 3: Build the Express Server

Create `server.js` with:
- Express app setup with middleware (cors, express.json, morgan)
- MongoDB connection with Mongoose
- Request logging middleware
- Error handling middleware
- API endpoints for all data types
- Health check and info endpoints

Step 4: Create CSV Import Utility

Build the import script to:
- Read CSV files from the archive directory
- Convert CSV data to JSON using csvtojson
- Transform data (handle `\N` null values, parse dates/numbers)
- Batch insert large datasets
- Track import statistics

Step 5: Set Up Logging

Configure Winston loggers:
- `logger` - general application logs
- `apiLogger` - HTTP request logs
- `dbLogger` - database connection logs
- `importLogger` - CSV import progress logs

Step 6: Acquire and Prepare Data

Obtain F1 CSV data from public datasets (e.g., Ergast API), place in `archive/` folder with files:
- drivers.csv
- constructors.csv
- circuits.csv
- races.csv
- results.csv
- driver_standings.csv
- constructor_standings.csv
- etc.

Step 7: Build the Angular Frontend

Create Angular project:

```bash
cd frontend
ng new f1-frontend
npm start
```

Build components for:
- Driver listing
- Constructor listing
- Race schedule
- Results display
- Championship standings

Step 8: Connect Frontend to Backend

Update Angular services to call the REST API:

```typescript
@Injectable({ providedIn: 'root' })
export class F1ApiService {
  private apiUrl = 'http://localhost:3000/api';

  getDrivers(): Observable<Driver[]> {
    return this.http.get<Driver[]>(`${this.apiUrl}/drivers`);
  }

  getRaces(year?: number): Observable<Race[]> {
    return this.http.get<Race[]>(`${this.apiUrl}/races`, {
      params: { year: year.toString() }
    });
  }
}
```

Step 9: Test and Debug

Run the server:
```bash
npm start
```

Import data:
```bash
npm run import
```

Test API endpoints with curl or Postman:
```bash
curl http://localhost:3000/api/drivers
curl http://localhost:3000/api/latest-results
```

Step 10: Production Build

Build the frontend for production:

```bash
npm run build
```

Set environment:

```bash
NODE_ENV=production npm start
```

Getting Started

Prerequisites

- Node.js (v14+)
- MongoDB (v4+)
- npm or yarn

Installation

1. **Clone the repository**

2. **Install backend dependencies**
   ```bash
   cd backend
   npm install
   ```

3. **Configure MongoDB**
   - Ensure MongoDB is running locally on `localhost:27017`
   - Or set `MONGO_URI` environment variable

4. **Import the data**
   ```bash
   npm run import
   ```

5. **Start the server**
   ```bash
   npm start
   ```

Run Frontend (Development)

```bash
cd frontend
npm start
```

Visit `http://localhost:4200`

Available Scripts

Command	Description
`npm start`	Start the backend server
`npm run import`	Import CSV data to MongoDB
`npm run frontend`	Start Angular dev server
`npm run build`	Build Angular for production

Future Enhancements

- [ ] User authentication and authorization
- [ ] Race simulation feature
- [ ] Driver and constructor comparisons
- [ ] Interactive circuit visualization
- [ ] Real-time race updates via WebSocket
- [ ] Mobile-responsive design
- [ ] Driver career statistics charts
- [ ] Constructor championship standings
- [ ] Lap time analysis
- [ ] Pit stop analytics

License

This project is for educational purposes (FET Subject).
